

RELATÓRIO TEÓRICO

Cloud DevOps: Orchestrating Containers and Microservices

1. Arquitetura de Microsserviços e o Papel do DevOps em Ambientes Cloud-Native

A arquitetura de microsserviços representa um paradigma de desenvolvimento de software que estrutura uma aplicação como um conjunto de serviços pequenos, independentes e fracamente acoplados. Cada serviço é responsável por uma capacidade de negócio específica, possui seu próprio banco de dados e se comunica com outros serviços através de APIs bem definidas, tipicamente utilizando protocolos HTTP/REST ou mensageria assíncrona. Esta abordagem contrasta fundamentalmente com a arquitetura monolítica tradicional, onde toda a funcionalidade da aplicação reside em uma única base de código e é implantada como uma unidade indivisível.

O movimento DevOps emergiu como resposta à necessidade de reduzir a fricção entre desenvolvimento (Dev) e operações (Ops), promovendo uma cultura de colaboração, automação e entrega contínua de valor. Em ambientes cloud-native, o DevOps assume um papel ainda mais crítico, pois a complexidade inerente a sistemas distribuídos exige práticas operacionais sofisticadas. A cultura DevOps implementa princípios como automação de pipelines, infraestrutura como código, monitoramento contínuo e feedback rápido, permitindo que equipes entreguem software de alta qualidade com maior frequência e menor risco.

A metodologia dos Doze Fatores (12-Factor App) fornece diretrizes essenciais para construção de aplicações cloud-native. Estes princípios incluem: base de código único rastreado em controle de versão; dependências explicitamente declaradas; configuração armazenada em ambiente; serviços de apoio tratados como recursos anexados; estágios de build e run separados; processos stateless; vinculação de porta através de variáveis de ambiente; concorrência através do modelo de processos; descartabilidade para maximizar robustez; paridade entre ambientes de desenvolvimento e produção; logs tratados como streams de eventos; e execução de tarefas administrativas como processos one-off. A adoção destes princípios é fundamental para garantir que aplicações de microsserviços sejam verdadeiramente portáteis, escaláveis e resilientes em ambientes de nuvem.

2. Containerização: Docker e Kubernetes

2.1 Conceitos Fundamentais de Containerização

A containerização é uma tecnologia de virtualização que encapsula uma aplicação e todas as suas dependências em uma unidade isolada chamada contêiner.

Diferentemente das máquinas virtuais tradicionais, que incluem um sistema operacional completo, os contêineres compartilham o kernel do sistema operacional do host, resultando em inicialização mais rápida, menor consumo de recursos e maior portabilidade. O Docker popularizou esta tecnologia ao fornecer uma plataforma completa para criação, distribuição e execução de contêineres, estabelecendo padrões de fato para a indústria.

Um contêiner Docker é construído a partir de uma imagem, que é um template read-only contendo o sistema de arquivos, bibliotecas e configurações necessárias para executar a aplicação. As imagens são construídas em camadas, permitindo reutilização eficiente de componentes comuns. O Dockerfile é o arquivo de configuração que descreve os passos para construir uma imagem, incluindo a imagem base, instalação de dependências, cópia de arquivos e definição de comandos de inicialização. A adoção de práticas como multi-stage builds permite criar imagens otimizadas, onde estágios de build são descartados, resultando em imagens finais significativamente menores e mais seguras.

2.2 Docker vs Kubernetes: Complementaridade e Casos de Uso

Docker e Kubernetes servem propósitos complementares no ecossistema de containerização. O Docker é uma plataforma de contêineres focada na criação, execução e distribuição de contêineres individuais. É ideal para ambientes de desenvolvimento local, testes isolados e aplicações simples que não requerem orquestração complexa. O Docker Compose estende estas capacidades permitindo a definição e execução de aplicações multi-contêiner através de um arquivo YAML declarativo, facilitando o desenvolvimento local de sistemas compostos por múltiplos serviços.

Kubernetes, por outro lado, é uma plataforma de orquestração de contêineres projetada para ambientes de produção em escala. Ele gerencia automaticamente a implantação, escalonamento e operação de aplicações containerizadas através de um conjunto rico de primitivas: Pods para agrupar contêineres relacionados, Deployments para gerenciar réplicas e atualizações, Services para descoberta e balanceamento de carga, ConfigMaps e Secrets para gerenciamento de configuração, e Horizontal Pod Autoscalers para escalabilidade automática baseada em métricas.

Comparativo: Docker vs Kubernetes

Aspecto	Docker	Kubernetes
Escopo	Contêineres individuais	Orquestração de clusters
Ambiente	Desenvolvimento local, testes	Produção, escala empresarial
Escalonamento	Manual	Automático (HPA/VPA)

Alta disponibilidade	Limitada	Nativa (replicas, health checks)
Descoberta de serviço	Docker Compose (local)	Services, DNS interno
Complexidade	Baixa	Alta (curva de aprendizado)

3. Orquestração de Containers

A orquestração de contêineres é o processo automatizado de gerenciamento do ciclo de vida de contêineres em ambientes de produção. Kubernetes tornou-se o padrão de fato para orquestração, oferecendo um conjunto abrangente de funcionalidades que abordam os desafios de operar aplicações distribuídas em escala. Os conceitos fundamentais incluem Pods, a menor unidade implantável que encapsula um ou mais contêineres; Deployments, que gerenciam réplicas de Pods e estratégias de atualização; Services, que fornecem endpoints estáveis para acesso aos Pods; e Namespaces, que permitem isolamento lógico de recursos dentro de um cluster.

O escalonamento automático é uma das funcionalidades mais poderosas do Kubernetes. O Horizontal Pod Autoscaler (HPA) ajusta automaticamente o número de réplicas de um Deployment com base em métricas como utilização de CPU ou memória, enquanto o Vertical Pod Autoscaler (VPA) ajusta os requests de recursos dos contêineres. Em nível de infraestrutura, o Cluster Autoscaler adiciona ou remove nodes do cluster conforme a demanda. Esta capacidade de escalabilidade elástica é essencial para lidar com variações de tráfego, como campanhas promocionais de e-commerce, garantindo que recursos sejam alocados dinamicamente sem intervenção manual.

4. CI/CD em Ambientes Distribuídos

Continuous Integration (CI) e Continuous Delivery/Deployment (CD) são práticas fundamentais para equipes que adotam arquiteturas de microsserviços. CI envolve a integração frequente de código em um repositório compartilhado, onde cada integração é verificada por builds automatizados e testes. CD estende este conceito, automatizando a entrega de código aprovado para ambientes de staging e produção. Em conjunto, estas práticas reduzem drasticamente o tempo entre concepção de uma funcionalidade e sua disponibilização para usuários finais.

Pipelines de CI/CD para microsserviços apresentam desafios únicos. Cada serviço possui seu próprio ciclo de release independente, exigindo pipelines modulares e reutilizáveis. Estratégias de deploy como rolling updates permitem atualizações graduais sem indisponibilidade; blue-green deployments mantêm duas versões ativas para rollback instantâneo; e canary releases direcionam gradualmente tráfego para a nova versão, permitindo validação em produção com risco limitado. Ferramentas como GitHub Actions, GitLab CI e Azure DevOps fornecem plataformas robustas para

implementação destes pipelines, com suporte nativo para secrets management, caching de artefatos e ambientes de deploy.

5. Observabilidade: Métricas, Logs e Traces

Observabilidade é a capacidade de entender o estado interno de um sistema através de suas saídas externas. Em arquiteturas de microsserviços, onde requisições atravessam múltiplos serviços, observabilidade é essencial para diagnóstico de problemas, otimização de performance e compreensão do comportamento do sistema. Os três pilares da observabilidade são métricas, logs e traces distribuídos, cada um oferecendo uma perspectiva complementar sobre a saúde e performance da aplicação.

Métricas são medições numéricas coletadas em intervalos regulares, como contagem de requisições, latência percentílica e utilização de recursos. Sistemas como Prometheus coletam métricas através de endpoints de exposição, enquanto Grafana fornece visualização e alertas. Logs são registros de eventos que capturam informações detalhadas sobre o comportamento da aplicação, sendo agregados por ferramentas como ELK Stack (Elasticsearch, Logstash, Kibana) ou Loki. Traces distribuídos rastreiam o fluxo de requisições através de múltiplos serviços, identificando gargalos e falhas em sistemas complexos. OpenTelemetry emergiu como padrão unificado para instrumentação, fornecendo APIs e SDKs para coleta de traces, métricas e logs de forma vendor-agnostic.

6. Justificativa das Decisões Arquiteturais

A arquitetura proposta para a plataforma de pedidos da Loja Veloz fundamenta-se em princípios de resiliência, escalabilidade e manutenibilidade. A decomposição em microsserviços alinha-se com os domínios de negócio: Pedidos, Pagamentos e Estoque, permitindo que cada serviço seja desenvolvido, implantado e escalado independentemente. O padrão API Gateway centraliza questões transversais como autenticação, rate limiting e roteamento, simplificando a comunicação entre clientes e serviços backend.

A escolha do Kubernetes como plataforma de orquestração justifica-se pela necessidade de escalabilidade automática durante picos de tráfego, alta disponibilidade através de réplicas distribuídas, e padronização do ambiente de deploy. A implementação de HPA permite que cada serviço escale horizontalmente conforme demanda, enquanto readiness e liveness probes garantem que apenas instâncias saudáveis recebam tráfego. A estratégia de deploy rolling update minimiza riscos durante atualizações, mantendo versões anteriores disponíveis até que a nova versão seja completamente validada.

O pipeline de CI/CD estabelece um fluxo automatizado desde commit até produção, com estágios de build, testes unitários, testes de integração, scan de vulnerabilidades e deploy progressivo. A instrumentação com OpenTelemetry fornece observabilidade end-to-end, permitindo identificação rápida de anomalias e diagnóstico de incidentes.

Esta combinação de práticas e tecnologias forma uma base sólida para operação confiável de uma plataforma de e-commerce de médio porte.

VIDEO DE APRESENTACAO

Assista ao video pitch de apresentacao deste trabalho:

<https://youtu.be/VXn-jSdxflY>